TECNOLÓGICO UNIVERSITARIO PLAYACAR · 7MO CUATRIMESTRE

Sistema de Gestión de Envíos "Riviera Express"

Desarrollo y Programación Orientado a Objetos 27-1 · Caso de Estudio #8

INTEGRANTES DEL EQUIPO

- Angel Ricardo Teh Catzin — **Matrícula: 7433**
- Alejandro Ramirez Perez — **Matrícula: 7507**
- Ever Lopez Panti — **Matrícula: 7335**
- Cristian Alejandro Hoil Reyes — **Matrícula: 7243**

DATOS ACADÉMICOS

Carrera: Ing. Sistemas Computacionales

Docente: Emmanuel Andres Martinez Lopez

Arquitectura: 5 Clases (.py) + 1 Main (.py)

Fecha: Noviembre 2026

Planteamiento del Problema y Solución

Problemática en "Riviera Express"

"Riviera Express" es una empresa de envíos urbanos que transporta paquetes y documentos pequeños, pero presenta tres problemas operativos:

- Cálculo manual de tarifas según **peso, volumen y distancia**.
- Riesgo de capturar **pesos o medidas negativas** que alteran los cobros.
- Falta de control para **agrupar varios paquetes en una hoja de ruta** y asignar un **repartidor responsable**.

Alcance de Nuestra Solución POO

- **Herencia:** Superclase `Envio` con subclases `Paquete` y `Documento`.
- **Encapsulamiento:** Protección contra números negativos usando `get/set` y `@property`.
- **Sobrecarga de 4 Operadores:** Uso directo de `+`, `-`, `*` y `/` entre objetos.
- **Agrupación y Asignación:** Clase `HojaRuta` vinculada a `Repartidor`.
- **Ciclo de Vida y Limpieza:** Constructores `__init__`, destructores `__del__`, sin comentarios ni excepciones.

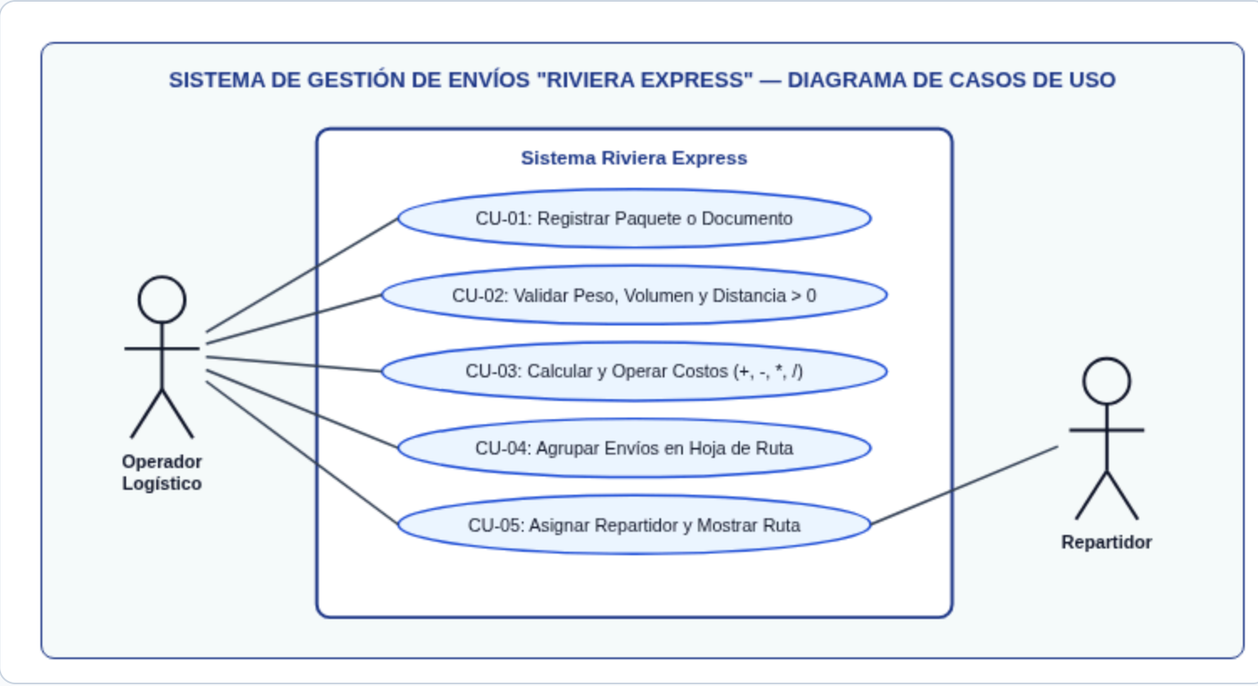
Especificación de Requisitos (RF y RNF)

Código	Requisito	Descripción	Aplicación en POO
RF-01	**Registro de envíos**	Registrar paquetes (con volumen) y documentos (con urgencia).	**Herencia** desde la clase padre `Envio`.
RF-02	**Validar medidas > 0**	Impedir que se asignen pesos, distancias o volúmenes negativos.	**Encapsulamiento** con `get/set` y `@property`.
RF-03	**Cálculo de costo**	Calcular tarifa según peso ($10/kg), distancia ($5/km) y volumen/urgencia.	Método `calcular_costo()` reutilizando `super()`.
RF-04	**Operar costos (+, -, *, /)**	Sumar, restar, multiplicar y dividir costos entre dos objetos de envío.	**Sobrecarga:** `__add__`, `__sub__`, `__mul__`, `__truediv__`.
RF-05 / 06	**Hoja de Ruta y Repartidor**	Agrupar varios envíos en una ruta y asignarle un repartidor responsable.	Clases `HojaRuta` y `Repartidor` conectadas.
RNF-01 / 04	**Arquitectura y Restricciones**	5 clases en archivos individuales + `main.py`, sin comentarios ni excepciones.	`1 Archivo = 1 Clase` (5 clases ≥ 4 integrantes).

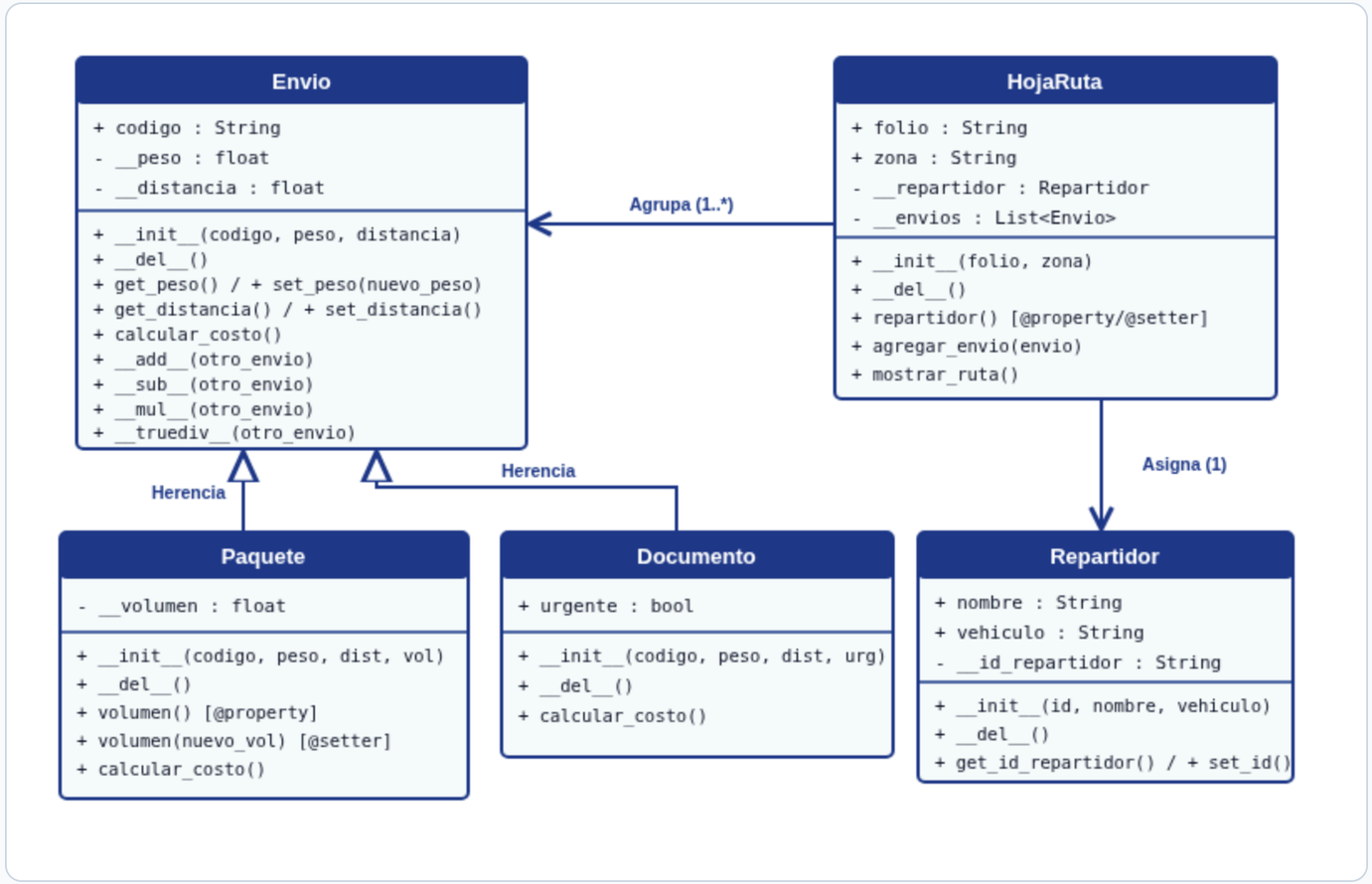
Análisis: Objetos Identificados y Casos de Uso

5 Clases Identificadas (≥ 4 Integrantes)

- **1. Envio (Superclase):** Atributos `+codigo`, `-__peso`, `-__distancia`. Calcula costo base y sobrecarga +, -, *, /.
- **2. Paquete (Subclase):** Hereda de `Envio`, encapsula `-__volumen` con `@property` y suma $20 por volumen.
- **3. Documento (Subclase):** Hereda de `Envio`, añade `+urgente` y suma $30 si es urgente.
- **4. Repartidor:** Guarda `+nombre`, `+vehiculo` y encapsula `-__id_repartidor` con `get/set`.
- **5. HojaRuta:** Agrupa la lista `-__envios` y asigna al `-__repartidor` mediante `@property`.



Diseño: Diagrama de Clases UML (+ Público / - Privado)



Herencia, Constructores/Destructores y Encapsulamiento

1. Superclase Envio.py (get y set)

```
class Envio:
    def __init__(self, codigo, peso, distancia):
        self.codigo = codigo
        self.__peso = peso
        self.__distancia = distancia

    def __del__(self):
        print(f"Envío {self.codigo} eliminado")

    def get_peso(self):
        return self.__peso

    def set_peso(self, nuevo_peso):
        if nuevo_peso > 0:
            self.__peso = nuevo_peso
        else:
            print("El peso no puede ser negativo")
```

2. Subclase Paquete.py (Herencia y @property)

```
from Envio import Envio

class Paquete(Envio):
    def __init__(self, codigo, peso, distancia, volumen):
        super().__init__(codigo, peso, distancia)
        self.__volumen = volumen

    def __del__(self):
        print(f"Paquete {self.codigo} eliminado")

    @property
    def volumen(self):
        return self.__volumen

    @volumen.setter
    def volumen(self, nuevo_volumen):
        if nuevo_volumen > 0:
            self.__volumen = nuevo_volumen
        else:
            print("El volumen no puede ser negativo")
```

Sobrecarga de 4 Operadores (+, -, *, /) y Hoja de Ruta

Sobrecarga en Envio.py (+, -, *, /)

```
    def calcular_costo(self):
        return (self.__peso * 10) + (self.__distancia * 5)

    def __add__(self, otro_envio):
        return self.calcular_costo() + otro_envio.calcular_costo()

    def __sub__(self, otro_envio):
        return self.calcular_costo() - otro_envio.calcular_costo()

    def __mul__(self, otro_envio):
        return self.calcular_costo() * otro_envio.calcular_costo()

    def __truediv__(self, otro_envio):
        return self.calcular_costo() / otro_envio.calcular_costo()
```

Agrupación en HojaRuta.py (@property)

```
class HojaRuta:
    def __init__(self, folio, zona):
        self.folio = folio
        self.zona = zona
        self.__repartidor = None
        self.__envios = []

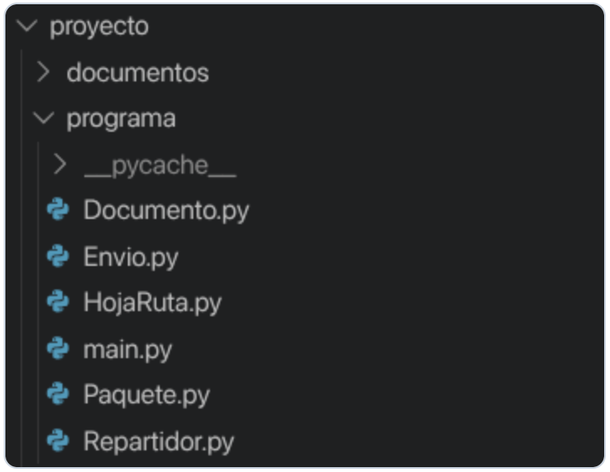
    @property
    def repartidor(self):
        return self.__repartidor

    @repartidor.setter
    def repartidor(self, nuevo_repartidor):
        self.__repartidor = nuevo_repartidor

    def agregar_envio(self, envio):
        self.__envios.append(envio)
```

Estructura Modular (1 Archivo = 1 Clase) y Ejecución de main.py

Estructura del Proyecto



Ejecución Limpia de main.py en Consola

